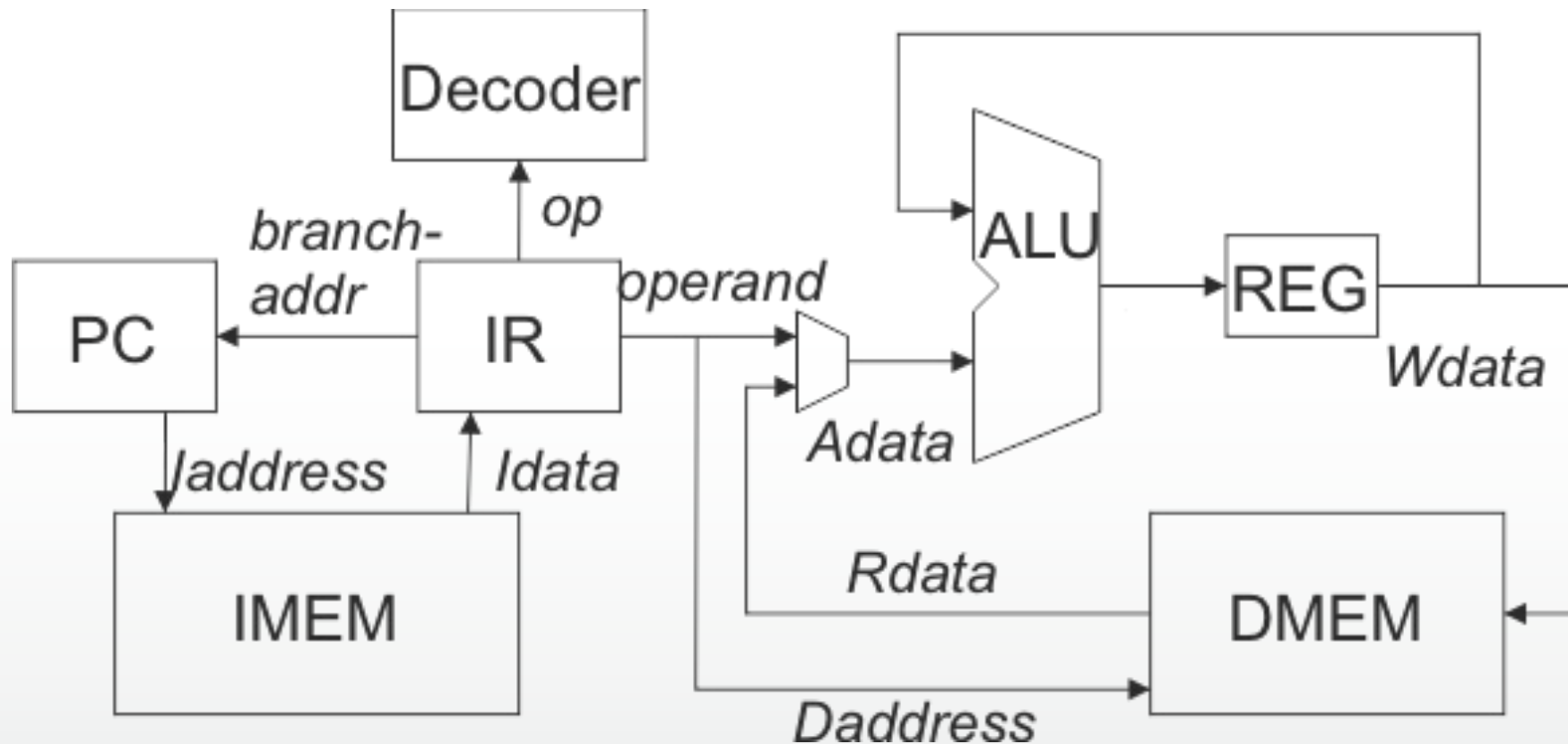


2. A Basic CPU

A very simple CPU



- Many similarities with RISC-V

- PC – Program Counter
- ALU – Arithmetic and Logic Unit
- REG – Register (Wdata)
- IR – Instruction Resolve
- Decoder – Instruction decoder
- IMEM – Instruction memory (read-only)
- DMEM – Data memory
- Note also the multiplexer between the IR and ALU

Harvard Architecture

- (Name goes back ~70 years!)
- In the Harvard Architecture, Instruction and Data memories are separate
- In Princeton or von Neumann architecture, single instruction+data memory
- In reality, modern CPUs are modified Harvard
 - Level 1 cache is divided between instructions and data
 - Level 2 cache and above is shared, single memory

Instructions

- C:

$a = b + c;$

- Assembly:

LD b

ADD c

ST a

- Machine Code (not RISC-V nor ARM)

LD b	000	00001
ADD c	010	00010
ST a	001	00011

Machine code

- Simple example:
 - 8 bit instructions
 - 3 bit opcode, 5 bit data

LD 000

ADD 010

ST 001

b 00001

c 00010

a 00011

- Note that a, b, and c are addresses in DMEM
- This is known as Direct mode

Immediate Mode

- Suppose we write in C:

`a = b + 2;`

- How do we know whether “2” is a number or an address?
- We use a different opcode:

`LD b            000 00001`

`ADDI 2         101 00010`

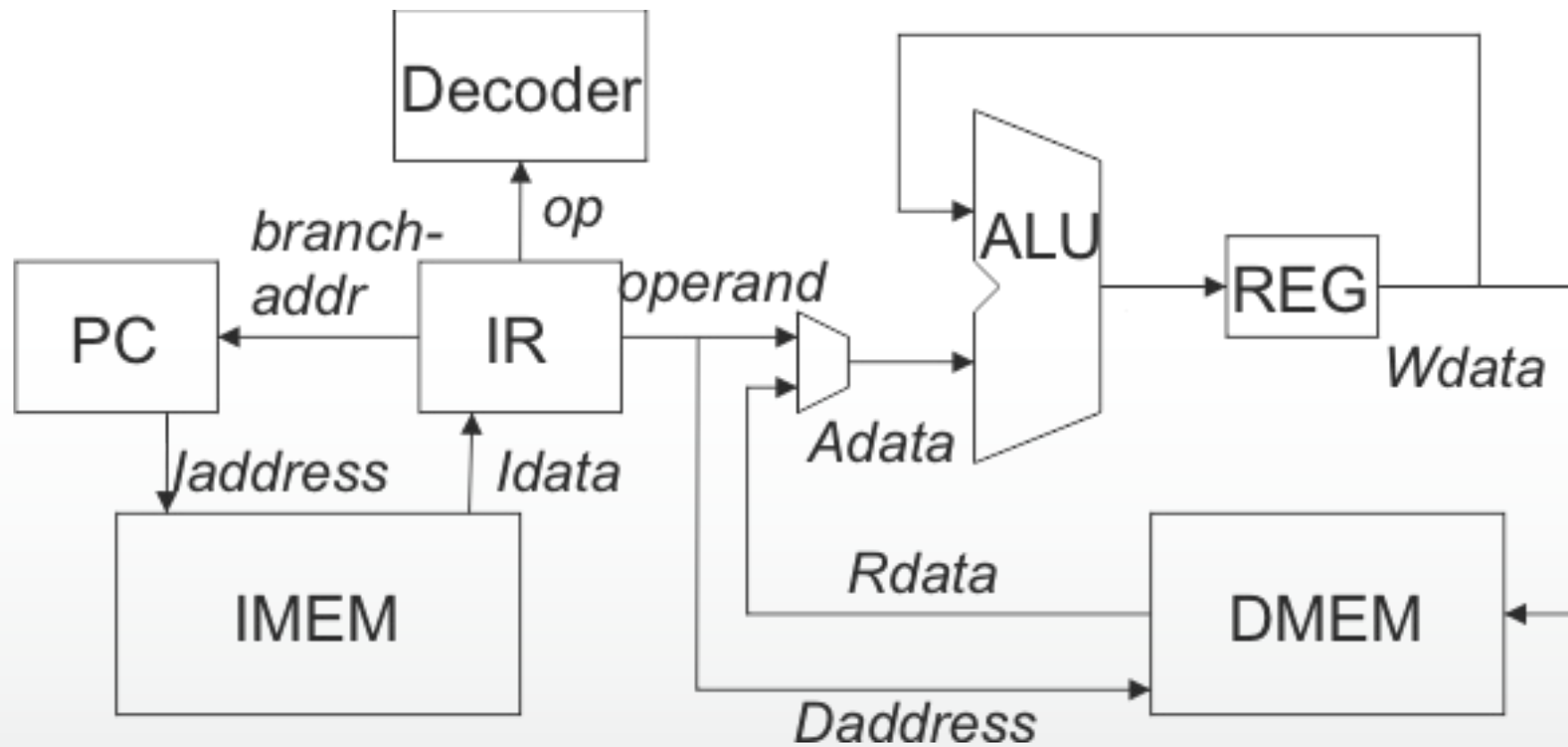
`ST a           001 00011`

- This is known as Immediate Mode
- The MUX in the CPU determines whether the ALU uses the operand (data) or data from the address given by the operand

A SystemVerilog Model of the CPU

- 150-200 lines of code
- Standard SystemVerilog constructs:
 - always_comb, always_ff
 - if, case
 - modules and instances
- Little here that you did not see last year

CPU



Fetch, Execute

- CPU takes 1 clock cycle for each instruction
 - Fetch new instruction from IMEM
 - Execute instruction
- Control signals are needed to send and receive data between the parts of the CPU

Control Signals

load_REG	Load REG
ALU_REG	Load REG with result from ALU
INC_PC	Increment PC, and save the result in PC
IMM	Select Immediate or Direct mode data
WE	Write Enable for DMEM
ALU_add	Perform an add operation in the ALU
ALU_sub	Perform a subtract operation in the ALU
z_flag	Set when result from ALU is zero

Opcodes

- opcodes.svh

```
`define LOAD 3'b000  
`define STORE 3'b001  
`define ADD 3'b010  
`define SUB 3'b011  
`define BNE 3'b100  
`define ADDI 3'b101
```

- SystemVerilog uses a back quote (`) where C uses #

Instruction Memory

```
module imem # (parameter WORD_W = 8, OP_W = 3)
    (input logic [WORD_W-OP_W-1:0] Iaddress,
     output logic [WORD_W-1:0] Idata);

`include "opcodes.svh"

always_comb
    case (Iaddress)
        0: Idata = {`STORE, 5'd30};
        1: Idata = {`LOAD, 5'd30};
        2: Idata = {`ADDI, 5'd2};
        3: Idata = {`STORE, 5'd30};
        4: Idata = {`BNE, 5'd1};
        default: Idata = 0;
    endcase
endmodule
```

Note that this module is parameterised.
By default, instructions and data are
8 bits (WORD_W).
Opcodes are 3 bits (OP_W).
Other modules are also parameterised.

Instruction Resolve

```
module ir    #(parameter WORD_W = 8, OP_W = 3)
              (input logic [WORD_W-1:0] Idata,
               output logic [WORD_W-OP_W-1:0] operand,
               output logic [OP_W-1:0] op);

logic [WORD_W-1:0] instr_reg;

always_comb //extract opcode and operand from instruction
    {op, operand} = Idata;

endmodule
```

Decoder Part 1

```
module decoder #(parameter OP_W = 3)
    (input logic z_flag,
     input logic [OP_W-1:0] op,
     output logic load_REG, ALU_REG, ALU_add,
                      ALU_sub, INC_PC, WE, IMM);

`include "opcodes.svh"

always_comb
    begin: com
        // reset control signals
        load_REG = 1'b0;  ALU_REG = 1'b0;
        ALU_add = 1'b0;   ALU_sub = 1'b0;
        WE = 1'b0;        IMM = 1'b0;
        INC_PC = 1'b1;
```

Decoder Part 2

```
unique case (op)
  `STORE: WE = 1'b1;
  `LOAD:  load_REG = 1'b1;
  `BNE:   if (!z_flag)
           INC_PC = 1'b0;
  `ADD:   begin
           load_REG = 1'b1;
           ALU_REG = 1'b1;
           ALU_add = 1'b1;
           end
  `SUB:   begin
           load_REG = 1'b1;
           ALU_REG = 1'b1;
           ALU_sub = 1'b1;
           end
```

```
  `ADDI:  begin
           IMM = 1'b1;
           load_REG = 1'b1;
           ALU_REG = 1'b1;
           ALU_add = 1'b1;
           end
           default::
           endcase
end

endmodule
```


ALU

```

module alu
    #(parameter WORD_W = 8)
    (input logic clock,
     n_reset, load_REG,
     ALU_REG, ALU_add,
     ALU_sub,
     input logic [WORD_W-1:0]
                     Adata,
     output logic [WORD_W-1:0]
                     Wdata,
     output logic z_flag);

    always_comb
        if (Wdata == 0)
            z_flag = 1'b1;
        else
            z_flag = 1'b0;

    always_ff @(posedge clock,
                negedge n_reset)
        begin
            if (!n_reset)
                Wdata <= 0;
            else
                if (load_REG)
                    if (ALU_REG)
                        if (ALU_add)
                            Wdata <= Wdata
                                + Adata;
                        else if (ALU_sub)
                            Wdata <= Wdata
                                - Adata;
                        else
                            Wdata <= Adata;
                end
            endmodule

```

4 control signals

1 status signal

PC- Program Counter

```
module pc #(parameter WORD_W = 8, OP_W = 3)
    (input logic clock, n_reset, INC_PC, load_PC,
     input logic [WORD_W-OP_W-1:0] branchaddr,
     output logic [WORD_W-OP_W-1:0] Iaddress);

    always_ff @(posedge clock, negedge n_reset)
        begin
            if (!n_reset)
                Iaddress <= 0;
            else
                if (INC_PC)
                    Iaddress <= Iaddress + 1;
                else
                    Iaddress <= branchaddr;
            end
        end
endmodule
```

Notice that the PC is incremented
in every execute cycle,
unless `z_flag == 1`

Data memory (dmem)

```
module dmem #(parameter WORD_W = 8, OP_W = 3)
    (output logic [WORD_W-1:0] Mdata,
     input logic [WORD_W-1:0] Wdata,
     input logic [WORD_W-OP_W-1:0] Daddress,
     input logic WE, clock);

logic [WORD_W-1:0] mem [2** (WORD_W-OP_W)];

always_ff @(posedge clock)
    if (WE)
        mem[Daddress] <= Wdata;

always_comb
    Mdata = mem[Daddress];

endmodule
```

CPU – Top Level

```
module cpu1 #(parameter WORD_W = 8, OP_W = 3)
    (input logic clock, n_reset,
     output logic [6:0] disp0, disp1);

logic load_REG, ALU_REG, ALU_add, ALU_sub,
    INC_PC, WE, IMM, z_flag;

logic [OP_W-1:0] op;
logic [WORD_W-OP_W-1:0] branchaddr, Iaddress, operand,
    Daddress;

logic [WORD_W-1:0] Idata, Wdata, Rdata, Adata, Mdata, digits;

pc #(WORD_W(WORD_W), OP_W(OP_W)) pc0 (.*) ;
imem #(WORD_W(WORD_W), OP_W(OP_W)) imem0 (.*) ;
ir #(WORD_W(WORD_W), OP_W(OP_W)) ir0 (.*) ;
decoder #(OP_W(OP_W)) decoder0 (.*) ;
alu #(WORD_W(WORD_W)) alu0 (.*) ;
dmem #(WORD_W(WORD_W), OP_W(OP_W)) dmem0 (.*) ;
```

CPU – Top Level (2)

```
always_comb  
  begin  
    branchaddr = operand;  
    Daddress = operand;  
    if (IMM)  
      Adata = {{OP_W{1'b0}}, operand};  
    else  
      Adata = Rdata;  
    end
```

```
always_comb  
  Rdata = Mdata;  
  
endmodule
```

This could be included in the other
always_comb block